Universidad Peruana Cayetano Heredia
Facultad de Ciencias e Ingeniería
Departamento de Ciencias Exactas

Programación Avanzada 2025 – Ingeniería Biomédica

Lab. 6.2. POO. Ocultamiento de Información. Métodos get() y set()

Septiembre 26, 2025

Cree una carpeta (folder) en el disco D, nómbrela con su apellido paterno seguido de su código. Ejemplo: LOPEZ12345

Programación Orientada a Objetos (POO)

Es un paradigma de programación que permite desarrollar programas basados en la forma que las personas conciben los objetos que nos rodean en el mundo real. Las personas piensan en forma natural sobre las cosas como objetos que tienen atributos y comportamientos, y éstos determinan la forma como interactuamos con ellos. La interacción se lleva a cabo en la mayoría de los casos en forma abstracta.

Interacción Abstracta

Si queremos cambiar el canal de televisión desde nuestros asientos, usamos el control remoto. El control remoto es un **objeto** con un número de **atributos** y **comportamientos** ocultos dentro de él. Sin entender los atributos ocultos (microchip, cableado, etc.), aun podemos saber y esperar que presionando un botón se ejecutará una particular función. En este caso la interacción con el control remoto es *en forma abstracta*. En esto reside la belleza de la POO – el foco está en cómo los objetos se comportan, no en el código requerido para decirles cómo comportarse.

Clases y Objetos

La *Programación Orientada a Objetos* (POO) *encapsula* los datos (*atributos*) y funciones (*comportamientos*) en componentes llamados *clases*. Los datos y funciones de una clase están íntimamente ligados. Una clase es como un plano. Usando un plano, un constructor puede construir una casa. Usando una clase, un programador puede crear un *objeto* (también llamado una *instancia*). Un plano puede ser usado varias veces para construir varias casas. Una clase puede ser usada varias veces para crear varios objetos de la misma clase. Las clases tienen una propiedad llamada *ocultamiento de información*. Esto significa que, aunque los objetos pueden saber cómo comunicarse el uno al otro a través de *interfaces* bien definidas, un objeto normalmente no está permitido saber cómo está implementado otro objeto – los detalles de implementación están ocultos dentro del objeto mismo.

A continuación, se muestra la clase `Account` que modela una cuenta bancaria.

```
class Account:
    def __init__(self, name, account_number, initial_amount):
        self.name = name
        self.no = account_number
        self.balance = initial_amount

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount, limit=3000):
        if(self.balance - amount >=0 and amount<=limit):
            self.balance -= amount
        else:
            if(self.balance - amount <0):
                print("No tiene suficiente dinero en la cuenta")
            if(amount>limit):
                print("No puede retirar mas que el límite diario:",limit)

    def dump(self):
        print(self.name, self.no, "balance:",self.balance)
```

1. Descargue el archivo `E1_Account.py` y guárdelo en vuestra carpeta de trabajo. Observe la clase `Account`, ¿qué puede decir acerca del diseño?, ¿representa adecuadamente a una cuenta bancaria?

2. ¿Qué sucedería si la clase `Account` no tuviera un inicializador (constructor)?.

3. Escriba un programa para probar la clase `Account`, guárdelo en un archivo llamado `E3_TestAccount.py`.

   a) Crear al menos dos objetos `Account`.
   b) Pruebe los métodos `deposit()`, `withdraw()` varias veces.
   c) Para uno de los objetos, modifique el nombre, el número de cuenta y el balance.

Ocultando las variables de instancia (campos de datos). Métodos get() y set().

En el ejercicio N° 3, vimos que es fácil modificar las variables de instancia, es decir los datos guardados, sin embargo, la Ingeniería de Software nos indica que es recomendable restringir el acceso a las variables de instancia. Esto debido a que muchos datos guardados tienen carácter sensible, por ejemplo, en la clase `Acount`, no sería pertinente que cualquiera pueda cambiar arbitrariamente el balance de una cuenta bancaria.

La mayoría de lenguajes orientados a objetos, tienen técnicas especializadas para restringir el acceso a las variables de instancia, sin embargo, con Python es necesario tener mayor cuidado, debido a su flexibilidad.

Python restringe el acceso a las variables de instancia modificando los nombres de las variables, anteponiendo a ellas **dos guiones bajo** o **un guion bajo**. Por ejemplo, las variables de instancia de la clase `Account` podrían tener la siguiente forma:

```
self.__name = name
self.__no = account_number
self.__balance = initial_amount
```

Si llevan dos guiones bajos se llaman **campos privados**, si llevan un solo guion bajo se llaman **campos protegidos** y si no llevan guion bajo se llaman **campos públicos**. Si se declaran los campos como privados, solo los métodos dentro de la clase deberían de tener acceso a esos campos (variables), desde el exterior no deberían tener acceso a dichas variables. A fin de que los objetos puedan modificar sus propias variables privadas de instancia, es necesario proporcionar los métodos `get()` para extraer los datos y `set()` para modificar los datos.

4. Escriba una nueva versión de la clase `Account`, llámela `Account2`, en la que las variables de instancia son privadas. Agregue los correspondientes métodos `get()` y `set()`.

5. Escriba un programa para probar la clase `Acount2`. Cree dos objetos `Account2`, para uno de los objetos intente mostrar los campos `name`, `no`, `balace` sin usar ningún método. Modifique los campos `name`, `no`, `balace` usando el método `set()`.

6. Elabore una nueva versión de la clase `Account2`, llamada `Account3`, en la que los objetos tengan la habilidad de mostrar la última transacción realizada por la cuenta que representan. Sugerencia: para cada cuenta, guarde los retiros y depósitos en una lista (guarde los montos de retiros y depósitos). Para mostrar la última transacción, bastaría mostrar el último valor ingresado a la lista.

7. Elabore un programa para probar la clase `Account3`.

8. Elabore una nueva versión de la clase `Acount3`, llamada `Acount4`, en la que cualquier objeto tenga la habilidad de mostrar la cantidad total de objetos creados. Preste atención al concepto de *variable de clase*.

9. Elabore un programa para probar la clase `Account4`.

10. Elabore, en el cuaderno, el diagrama de clases UML para la clase Acount2.

Ejercicio a resolver en casa:

11. Elabore un programa que realice de manera consecutiva:
    - Generar N objetos `Acount4`, con valores aleatorios
    - Permita que cada objeto realice o bien un depósito o bien un retiro.
    - Guarde en un archivo todos los objetos creados.
    - Elimine de la memoria los objetos creados.
    - Lea el archivo guardado.
    - Indique a uno de los objetos que muestre la cantidad actual de objetos en memoria.
    - Imprima el dinero actual de cada cuenta.

12. Diseñe una clase llamada `Stock` que representa las acciones de una empresa y que contenga:

    - Un campo privado del tipo string llamado `symbol` para el símbolo de la acción.
    - Un campo privado del tipo string llamado `name` para el nombre de la acción.
    - Un campo privado del tipo float llamado `previousPrice` que guarda el precio de la acción del día anterior.
    - Un campo privado del tipo float llamado `currentPrice` que guarda el precio de la acción del día.
    - Un inicializador (constructor) que crea una acción con símbolo, nombre, previo precio, y precio actual específicos.
    - Un método `get()` para retornar el nombre de la acción
    - Un método `get()` para retornar el símbolo de la acción
    - Los métodos `get()` y `set()` para extraer/modificar el precio previo de la acción
    - Los métodos `get()` y `set()` para extraer/modificar el precio actual de la acción
    - Un método llamado `getChangePercent()` que retorna el porcentaje de la variación del precio previo al actual.

Elabore el Diagrama UML para la clase `Stock` y luego implemente la clase.
Escriba un programa para probar la clase. Cree un objeto Stock con símbolo INTC, con nombre Intel Corporation, con precio previo de 80.5, y con precio actual de 80.75, y muestre el porcentaje del cambio del precio.

Guarde todos vuestros programas en una carpeta con el nombre su **Apellido** paterno seguido de vuestro **DNI**, luego comprima esta carpeta. Envíe este archivo a Katherine Navarro katherine.navarro@upch.pe especificando como asunto **Lab6.2**.